Reducing Exploration Cost in OpenMP Scheduling Algorithm Selection

Shaan Chopra (240905140), Mrinal Dhami (240905026),
Aniket Verma (240905156)

School of Computer Science and Engineering
Manipal Institute of Technology

Group 3

Abstract. OpenMP assigns the iterations of a parallel loop to worker threads according to a scheduling algorithm. Static scheduling has little dispatch overhead but can leave threads idle on irregular loops; fine-grained dynamic scheduling can balance work but introduces synchronization and locality costs. Recent work therefore treats scheduling as an online algorithm-selection problem. A 2025 study integrated expert selectors, Q-learning, and SARSA into LB4OMP and showed that reinforcement-learning selectors can discover high-performing schedules. Their explore-first policy, however, evaluates all 144 ordered state–action combinations for a 12-algorithm portfolio. In a 500-step application, the learning phase alone occupies 28.8% of loop instances.

This project investigates whether cheap loop features and prior measurements can remove most of that exploration tax. We propose a warm-started contextual bandit that observes iteration count, thread count, a STATIC timing probe, per-thread imbalance, and recent timing variation. A disjoint LinUCB policy ranks a reduced, training-only portfolio of representative LB4OMP algorithms; the expert *expChunk* rule limits pathological fine-grained trials. The selector is evaluated on Mandelbrot, STREAM Triad, and triangle counting using scenario-wise holdouts. It is compared with STATIC, GSS, ExhaustiveSel, explore-first Q-learning, a context-free bandit, and an offline Oracle. Primary outcomes are cumulative normalized regret, loop instances to reach within 5% of Oracle, total execution time, and selector overhead. The project is designed to test a falsifiable claim: informed bandit selection can approach Oracle in substantially fewer than 144 loop instances without sacrificing steady-state performance.

Keywords: OpenMP · Dynamic Loop Scheduling · Contextual Bandits · LinUCB · Load Balancing · High-Performance Computing

1 Introduction

OpenMP provides a portable shared-memory programming interface for C, C++, and Fortran. In a worksharing loop, a scheduler divides N logical iterations among P threads. The scheduling decision is performance-critical because the cost of an iteration may be uniform, spatially irregular, or may change between successive time-steps. The OpenMP `schedule` clause exposes standard policies such as `static`, `dynamic`, and `guided` [1].

With `static`, the assignment is computed in advance. It preserves locality and requires almost no synchronized dispatch, but it performs poorly when equal iteration counts do not represent equal work. At the other extreme, self-scheduling (SS) can issue one iteration whenever a thread requests work. This reduces idle time but can make queue synchronization and cache-line movement more expensive than useful computation. Guided and adaptive methods choose decreasing or performance-weighted chunk sizes to trade scheduling overhead for balance [2, 3].

LB4OMP extends LLVM's OpenMP runtime with a portfolio of static, dynamic, and adaptive loop schedulers [3]. No member of this portfolio dominates for every loop and machine. The situation is an instance of Rice's algorithm selection problem: infer a mapping from a problem-and-system description to the algorithm expected to perform best [6].

The research question of this project is:

> Can a feature-informed, warm-started contextual bandit select an OpenMP loop scheduler within 5% of an offline Oracle in far fewer loop instances than the 144-instance explore-first Q-learning baseline?

The project contributes (i) a reproducible LB4OMP measurement pipeline on a single multicore system, (ii) a leakage-safe warm-start procedure using held-out workload scenarios, (iii) a low-overhead LinUCB selector coupled with the *expChunk* rule, and (iv) an evaluation that separates exploration cost, steady-state quality, decision overhead, and robustness to workload drift.

1.1 Research Hypotheses

H1—exploration. FeatureBandit will reach a rolling mean loop time within 5% of Oracle in at most 30 loop instances for most held-out scenarios, compared with the fixed 144-instance learning phase of the published RL baseline.

H2—end-to-end performance. Over 500 repeated loop instances, FeatureBandit will reduce cumulative normalized regret and total loop time relative to explore-first Q-learning with a loop-time reward.

H3—role of design components. Removing the contextual warm start, portfolio reduction, or *expChunk* will measurably increase early-stage regret. These are hypotheses to be tested, not assumed results.

2 Reference Study and Current Research Gap

Müller Korndörfer et al. studied automated OpenMP scheduling selection using LB4OMP across six applications and three dual-socket CPU systems [5]. Their selectable portfolio contains the 12 algorithms in Table 1. Each loop is assigned a unique identifier, so a selector can maintain independent state whenever that loop reappears.

2.1 Methodology of the Reference Study

The study compares three expert-based selectors—RandomSel, ExhaustiveSel, and ExpertSel—with tabular Q-learning and SARSA. RandomSel changes algo-

Table 1: Scheduling portfolio used by the reference study and retained for the full-portfolio ablation.

Source/family	Algorithms	Main behavior
OpenMP standard	STATIC, SS, GSS	Precomputed equal chunks; one-at-a-time self-scheduling; decreasing guided chunks.
LLVM runtime	Auto(LLVM), TSS, Static Steal	Runtime-selected policy; trapezoidal decrease; static ownership with stealing.
LB4OMP	mFAC2, AWF-B, AWF-C, AWF-D, AWF-E, mAF	Factoring and adaptive weighted/factoring methods that revise chunk assignment from runtime behavior.

rithm with a probability derived from measured load imbalance. ExhaustiveSel tries every candidate once over successive time-steps, keeps the fastest, and can restart the search when imbalance changes. ExpertSel uses fuzzy rules over loop time and load imbalance. The RL agents instead learn values for transitions from a previous scheduler (state) to a next scheduler (action), with either loop time (LT) or load imbalance (LIB) supplying the reward.

The factorial experiment includes Mandelbrot, STREAM Triad, Triangle Counting, HACCKernels, LULESH, and SPHYNX Evrard collapse. It uses Broadwell (20 cores), Cascade Lake (56 cores), and AMD EPYC (128 cores), without hyperthreading. The authors evaluate default chunks and *expChunk*, two reward types, and five repetitions: 720 configurations and 3,600 executions in total. Oracle is constructed offline from exhaustive measurements and provides an unattainable comparison ceiling rather than a deployable policy.

The RL tables begin at zero and the explore-first policy exercises all $12 \times 12 = 144$ ordered combinations before exploitation. With 500 repeated loop instances, exploration consumes

$$\frac{144}{500} \times 100 = 28.8\%. \tag{1}$$

The study reports that LT is generally the more useful reward. LIB may favor SS because it balances finishing times, even when its synchronization and locality costs make total execution slower. The *expChunk* parameter improves the methods in many cases and reduces the damage caused by pathological trials. ExhaustiveSel with *expChunk* is the most consistently close to Oracle, while Q-learning demonstrates adaptability but pays the fixed exploration cost. The paper explicitly identifies alternative exploration, prior knowledge, transfer learning, and model-based methods as future directions.

2.2 Gap Addressed by This Project

Explore-first learning treats an easy regular loop and a highly uncertain irregular loop identically. It ignores information available before or during the first

execution: N, P, STATIC timing, per-thread finishing times, and recent drift. Its state–action table also spends trials learning transition orders even though the immediate objective is to choose the fastest current scheduler. Finally, the exploration budget grows quadratically with portfolio size in the reference formulation.

Our proposal changes the learning unit. A bandit round corresponds directly to one loop instance and one selected scheduler. Context predicts the immediate reward of each scheduler; only the selected scheduler's reward is observed. This matches the feedback available at runtime and targets cumulative execution cost rather than an indirect balance metric. Table 2 summarizes the methodological difference.

Table 2: Reference RL method versus the proposed method.

Aspect	Explore-first Q-learning	Proposed FeatureBandit
Decision model	Previous algorithm as state; next algorithm as action	Current loop/system feature vector; one reward model per action
Initial knowledge	All Q values set to zero	Training-only measurements form a regularized prior
Exploration	Fixed 144 ordered transitions	Optimism proportional to model uncertainty (LinUCB)
Portfolio	All 12 throughout learning	Training-only representative subset; full 12 used as ablation
Reward	Discrete LT or LIB feedback	Bounded, continuous normalized loop-time reward
Safety mechanism	Optional *expChunk*	*expChunk* enabled by default; extreme slowdown guard

3 Proposed System and Mathematical Formulation

Let $\mathcal{A}$ be a portfolio of schedulers. At repeated loop instance t, the selector observes a context vector $x_t \in \mathbb{R}^d$, chooses $a_t \in \mathcal{A}$, and measures parallel loop time $T_t(a_t)$. For comparison, the offline Oracle action is

$$a_t^* = \arg\min_{a \in \mathcal{A}} \text{median}_{r \in \mathcal{R}}\, T_{t,r}(a), \tag{2}$$

where $\mathcal{R}$ contains independent repetitions of the same controlled scenario. Oracle data are never visible to an online test selector.

The main reward is a bounded monotonic transformation of execution time:

$$r_t = \text{clip}\left(-\frac{\log(T_t(a_t)/T_{\text{ref}})}{\log 4}, -1, 1\right), \tag{3}$$

where T_{ref} is the initial STATIC probe time and remains fixed for that online run. Positive reward means faster than the reference. Clipping prevents a single interrupted or thermally throttled measurement from dominating the model.

Because Eq. 3 is monotonic in loop time, it does not repeat the LIB-reward failure mode.

3.1 Context Features

The core features in Table 3 require only loop metadata and timing already collected around the OpenMP barrier. They are standardized using training-fold statistics. No test scenario contributes to feature scaling or prior construction.

Table 3: Context available to the online selector.

Feature group	Values	Purpose
Loop scale	$\log_2 N$, P, N/P, loop identifier	Represent dispatch granularity and parallelism.
Initial probe	STATIC loop time, LIB, coefficient of variation of per-thread times	Estimate cost scale and irregularity from one useful execution.
Recent behavior	Last normalized time, moving mean and variance, change in LIB	Detect noise or phase changes.
System	Physical-core count and normalized thread count	Retain a small amount of machine context without a hardware-specific model.
Optional counters	Instructions, LLC misses, cycles per iteration	Tested only if unprivileged hardware counters are reliable; excluded from the core claim.

For finish times $f_{t,p}$ measured relative to the start of the parallel loop, load imbalance is recorded as in the reference work:

$$\mathrm{LIB}_t = \left(1 - \frac{\frac{1}{P}\sum_{p=1}^{P} f_{t,p}}{\max_p f_{t,p}}\right) \times 100. \tag{4}$$

LIB is a context and diagnostic metric, not the optimization target.

3.2 Warm-Started Disjoint LinUCB

LinUCB is selected as the primary bandit because it supports continuous rewards, has a closed-form update, exposes uncertainty explicitly, and requires only small $d \times d$ matrices [9]. For each action a, the disjoint model stores

$$A_a = \lambda I + \sum_{i:a_i=a} w_i x_i x_i^\top, \qquad b_a = \sum_{i:a_i=a} w_i r_i x_i. \tag{5}$$

The sums initially contain only offline measurements from training scenarios; $0 < w_i \leq 1$ limits the strength of transferred evidence. At loop instance t, the selector computes

$$\hat{\theta}_a = A_a^{-1} b_a, \tag{6}$$

$$p_{t,a} = \hat{\theta}_a^\top x_t + \alpha \sqrt{x_t^\top A_a^{-1} x_t}, \tag{7}$$

$$a_t = \arg\max_{a \in \mathcal{A}} p_{t,a}. \tag{8}$$

After observing r_t, it performs $A_{a_t} \leftarrow A_{a_t} + x_t x_t^\top$ and $b_{a_t} \leftarrow b_{a_t} + r_t x_t$. Hyperparameters λ, α, and prior weight w are selected on validation scenarios only. Thompson Sampling is retained as an optional secondary bandit, not part of the minimum successful scope [10].

3.3 Portfolio Reduction and Safety Rules

The 12 algorithms are represented by their training-only normalized performance vectors and grouped by agglomerative clustering. STATIC and GSS are always retained as interpretable anchors; one medoid from each remaining cluster is added. The target reduced size is six, with $K \in \{4, 6, 8\}$ selected during validation. This avoids choosing representatives from held-out test outcomes. The full 12-action LinUCB experiment isolates whether improvement comes from contextual exploration or merely from a smaller action set.

All non-STATIC algorithms use *expChunk* by default. If one loop instance exceeds $4T_{\text{ref}}$, the next instance falls back to the best safe arm among STATIC and GSS while the failed arm receives the clipped reward -1. If the five-instance moving median changes by more than 15%, exploration is temporarily increased rather than deleting the transferred prior. Thresholds are fixed before final testing and reported as a sensitivity analysis.

4 Workloads and Dataset Construction

The experiment uses three complementary workloads. Mandelbrot is a controllable compute-bound kernel whose pixel iteration counts create spatially irregular work. We implement constant, increasing, and decreasing imbalance profiles over 500 time-steps, following the intent of the reference study. STREAM Triad is a regular memory-bandwidth kernel and acts as a negative control: a selector should quickly avoid schedules whose dispatch or locality costs dominate useful work [11]. Triangle Counting from the GAP Benchmark Suite provides a sparse-graph workload in which vertex degrees create genuine data-dependent imbalance [12].

Each raw observation stores the fields in Table 4. A run manifest also records CPU model, core topology, RAM, OS/kernel, compiler, LLVM/LB4OMP commit, compilation flags, affinity variables, input hash, random seed, and timestamp. Raw measurements are never overwritten by aggregated results.

4.1 Leakage-Safe Warm-Start Split

Randomly splitting loop rows would be invalid: neighboring rows from the same scenario are almost identical. We therefore define a *scenario* as the application, input/profile, problem size, and thread count. All time-steps and repetitions of

Table 4: Structure of one loop-instance observation.

Field group	Stored values
Identity	Run, application, scenario, input/graph, loop ID, time-step, repetition
Configuration	N, P, scheduler, chunk, selector, seed, CPU affinity
Timing	Wall-clock loop time, selector decision time, per-thread start/finish time
Derived	LIB, coefficient of variation, normalized reward, phase/drift flag
System controls	CPU frequency/temperature when available, run order, build and software versions
Status	Success, timeout, validation failure, or outlier reason; raw value retained

one scenario remain together. Leave-one-scenario-out evaluation is used within each workload family, and a stronger leave-one-workload-out test is reported when enough training scenarios exist. Portfolio clustering, standardization, and prior fitting are repeated inside every training fold.

Offline labels are obtained by running each fixed scheduler on the training scenarios. These runs serve two purposes: estimate a warm-start prior and select representative arms. For a held-out scenario, the only initial information is its first STATIC execution and loop metadata. Oracle measurements for that scenario are opened only after selector logs are finalized.

5 Experimental Methodology

5.1 Implementation Plan

LB4OMP is built from a pinned public commit and compiled with one documented LLVM/Clang toolchain. The selector is inserted at the loop-level dispatch point used by the existing RL implementation. Timing uses a monotonic high-resolution clock outside the useful loop body, and per-thread finish times are collected in preallocated, cache-line-padded storage. Logging is buffered and written after the timed region. A replay mode feeds recorded contexts to the bandit so that policy logic can be unit-tested independently of OpenMP execution.

The implementation is validated in four stages: (1) serial output checks for Mandelbrot and STREAM; (2) triangle-count equality against GAP's reference result; (3) chunk-assignment sanity tests for every scheduler; and (4) timing instrumentation tests in which selector and logging costs are measured with an empty synthetic loop. Seeds and run manifests make every randomized decision replayable.

5.2 Experimental Factors

Table 5 defines the planned core matrix. The physical-core count P_{phys} is detected during system characterization; SMT is disabled or excluded from the primary analysis. If a machine has fewer than four physical cores, duplicate

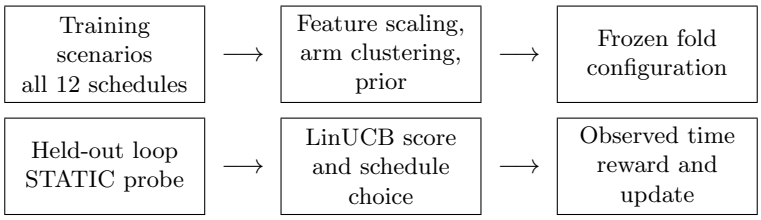


Figure 1: End-to-end workflow. Everything above the gap is fitted with training scenarios only. The lower row is the online path repeated for each loop instance of a held-out scenario.

thread-count levels are removed.

Table 5: Core factorial design for the mini-project.

Factor	Values
Workloads	Mandelbrot (constant/increasing/decreasing imbalance), STREAM Triad, GAP Triangle Counting (uniform and scale-free/Kronecker graphs)
Time horizon	500 loop instances for direct comparison; 50 and 200 instance truncations to study short applications
Threads	2, 4, and $\min(8, P_{\text{phys}})$, unique valid levels only
Fixed schedules	STATIC and GSS; all 12 schedules for offline Oracle
Selectors	ExhaustiveSel, Q-Learn-LT, context-free UCB1, FeatureBandit-K, FeatureBandit-12, Oracle
Chunk rule	*expChunk* primary; default chunks as an ablation
Repetitions	Seven independent runs per test cell after two untimed warm-ups
Primary split	Leave one scenario out; application-level holdout reported as a generalization stress test

UCB1 is included to determine whether the context and transferred prior add value beyond ordinary adaptive exploration [8]. Q-Learn-LT uses the published zero initialization, $\alpha = \gamma = 0.5$, reward constants, and explore-first policy whenever these are supported by the released code. Any necessary deviation is documented rather than silently approximated.

5.3 Noise Control and Execution Procedure

Benchmarks run on AC power with the machine otherwise idle. Threads are pinned to physical cores using `OMP_PLACES=cores` and `OMP_PROC_BIND=close`. The CPU governor, turbo state, NUMA placement, and SMT state are recorded and held constant within a campaign. Treatment order is randomized in blocks so that temperature or background activity does not systematically favor one selector. Two warm-up runs populate code and data pages; seven measured repetitions follow.

Timeouts and correctness failures are reported separately. Timing out an

arm is valid evidence of high cost for cumulative-regret calculations, where it is capped at the predeclared timeout; it is not silently discarded. Medians and interquartile ranges are the default summaries because laptop timing noise is rarely Gaussian.

6 Baselines and Evaluation Metrics

The Oracle is an offline lower bound computed from median fixed-scheduler runs for the same scenario and phase. STATIC and GSS test whether learning is useful at all. ExhaustiveSel represents lower-complexity expert search. Published Q-Learn-LT with *expChunk* is the direct baseline. Context-free UCB1 isolates the contribution of context, while FeatureBandit-12 isolates the contribution of portfolio reduction.

The primary metric is cumulative normalized regret through horizon H:

$$R_H^{\text{norm}} = \frac{\sum_{t=1}^{H} \big(T_t(a_t) - T_t(a_t^*)\big)}{\sum_{t=1}^{H} T_t(a_t^*)}. \tag{9}$$

This metric charges a selector for bad exploration and therefore reflects the end-to-end research goal. We additionally report:

- total application and loop time normalized to Oracle and to STATIC;
- *time-to-near-Oracle*: the earliest t_0 at which the trailing 20-instance mean is within 5% of Oracle and remains so for 30 instances;
- number of distinct arms tried and fraction of time spent away from the current empirically best arm;
- steady-state regret after instance 144, separated from learning-phase regret through instance 144;
- selector decision latency, logging overhead, and memory per loop ID;
- selected-arm accuracy and top-k hit rate against the offline ranking;
- robustness after a controlled workload phase change.

Seven paired repetitions are summarized with median and 95% bootstrap confidence intervals. Pairing uses the same scenario, thread count, and run block. For confirmatory comparisons we use a two-sided Wilcoxon signed-rank test on per-cell normalized runtime with Holm correction; effect sizes and intervals are emphasized over binary significance. H1 is supported only if the target is met in at least 75% of held-out scenarios and the upper bootstrap confidence bound on median convergence is below 144.

7 Ablation and Sensitivity Analysis

The following ablations make the explanation testable rather than attributing all gains to the complete system:

1. **No warm start:** set $A_a = \lambda I$ and $b_a = 0$ for every arm.

2. **No context:** replace LinUCB with UCB1 over measured rewards.
3. **Full portfolio:** retain all 12 arms with the same prior.
4. **Default chunks:** disable *expChunk* to quantify its safety and locality benefit.
5. **Reward:** compare normalized loop time with LIB reward to attempt replication of the reference paper's negative result.
6. **Probe size:** compare one initial STATIC observation with the median of three probes.
7. **Drift response:** disable phase-change detection on increasing and decreasing Mandelbrot profiles.

Sensitivity plots vary α, prior weight w, reduced portfolio size K, and drift threshold over values chosen before the final test run. These plots will reveal whether improvement depends on one fragile parameter setting.

8 System Workflow and Deliverables

The project is divided into four reproducible layers. The benchmark layer contains fixed inputs and correctness checks. The runtime layer contains LB4OMP and selector changes. The experiment layer expands a declarative configuration into pinned benchmark runs and immutable CSV logs. The analysis layer reads only those logs and produces tables, confidence intervals, selection traces, and regret curves.

Planned deliverables are: a pinned LB4OMP fork or patch set; implementations of Q-Learn-LT, UCB1, and FeatureBandit behind the same interface; benchmark sources and inputs; a run manifest and automation scripts; raw tidy-format measurements; an analysis notebook/script; and a final report containing both successful and failed hypotheses. The code repository will include a minimal command that reproduces one complete scenario before the full campaign is attempted.

9 Compute Requirements and Feasibility

The main experiments need only a uniform multicore CPU; a GPU is unnecessary. The exact number of physical cores is treated as measured configuration rather than assumed hardware. The lower-cost core campaign uses three thread levels, seven scenario families, six non-oracle selectors, and seven repetitions, yielding approximately

$$3 \times 7 \times 6 \times 7 = 882 \tag{10}$$

selector runs before ablations. Oracle construction adds at most 1,260 fixed-scheduler runs. Jobs are independent and can run overnight in randomized blocks. A pilot measures wall time before the matrix is launched; if necessary, input sizes are scaled so one 500-instance run remains below five minutes without changing the number of time-steps.

Table 6: Indicative six-week execution plan.

Week	Milestone and acceptance check
1	Build LLVM/LB4OMP; run all 12 schedules on a synthetic loop; freeze software versions and system manifest.
2	Implement/integrate workloads, timing, per-thread instrumentation, and correctness tests; produce raw CSV for one pilot scenario.
3	Reproduce fixed-schedule rankings and explore-first Q-learning behavior; verify that 144 exploration instances are logged.
4	Implement UCB1, feature pipeline, LinUCB prior/update, arm clustering, and replay tests; freeze hyperparameters on validation scenarios.
5	Execute randomized, blocked experiments and ablations; audit missing runs, timeouts, thermal drift, and output correctness.
6	Compute intervals/tests, create regret and selection plots, interpret failure cases, archive artifacts, and write the final report.

LinUCB itself is small: with at most 12 arms and fewer than 16 core features, the model stores only a few thousand floating-point values. Rank-one updates are $O(d^2)$ if the inverse is maintained, which should be negligible beside a meaningful parallel loop. Decision overhead is nevertheless measured rather than assumed negligible.

10 Threats to Validity and Mitigation

External validity. One consumer or laboratory CPU cannot establish performance across all HPC systems. We therefore restrict the claim to the measured architecture and treat cross-thread-count tests as limited portability evidence. Scripts and manifests permit later replication elsewhere.

Benchmark representativeness. Three kernels do not cover all OpenMP applications. They are chosen to span regular memory-bound, controlled compute-bound irregular, and data-dependent graph behavior. Conclusions will name these classes rather than generalizing to every loop.

Oracle instability. The fastest measured arm can change due to noise. Oracle uses medians from repeated fixed-schedule runs, and gaps are reported with uncertainty. Near-ties are treated as an equivalence set within the confidence interval rather than as definite misclassifications.

Temporal non-stationarity. A warm prior learned on one phase may become misleading. Controlled Mandelbrot phase changes, moving context, and a drift ablation quantify this risk.

Measurement perturbation. Per-thread timers, counters, and logging may alter short loops. Storage is preallocated and padded, I/O occurs outside the timed region, and empty-loop instrumentation overhead is measured and published. Problem sizes are selected so useful loop time comfortably exceeds measurement resolution.

Data leakage. Arm selection and warm-start fitting can leak test performance

if row-level splits or global clustering are used. The entire pipeline—scaling, clustering, hyperparameter tuning, and prior construction— is refit within the training portion of each scenario-wise fold.

11 Expected Contribution and Success Criteria

The expected contribution is not a new OpenMP scheduling primitive. It is an empirical answer to a sharply scoped systems question: how much of the RL exploration cost can be removed by using cheap context and prior experience? The primary success condition is a statistically supported reduction in time-to-near-Oracle and cumulative regret relative to Q-Learn-LT, without a material increase in steady-state runtime or selector overhead. A secondary success is showing which component—warm start, context, arm reduction, or *expChunk*—accounts for the improvement.

The project remains informative if H1 fails. Failure may show that STATIC imbalance features do not predict scheduler rankings, that transferred priors are architecture-specific, or that reduced portfolios omit rare but important arms. Because the protocol retains selection traces and a full-portfolio ablation, these explanations can be distinguished instead of reporting only an aggregate slowdown.

12 Conclusion

OpenMP loop scheduling is a balance between dispatch overhead, data locality, and idle time, and the best balance depends on the loop and machine. The reference study demonstrates that runtime RL selection is viable but also makes the main limitation measurable: an uninformed 144-instance exploration phase can consume 28.8% of a 500-step run. This synopsis proposes a compact response suited to a mini research project.

The planned selector uses an initial STATIC measurement and recent execution features to warm-start a disjoint LinUCB policy. A training-only reduced portfolio and *expChunk* control the cost of unsafe trials. Scenario-wise holdouts, an offline Oracle, direct Q-learning reproduction, cumulative regret, convergence time, and targeted ablations turn the idea into a falsifiable experiment. The result will establish whether feature-informed bandit selection can preserve RL's adaptability while making online OpenMP schedule selection useful for shorter executions on modest multicore hardware.

References

[1] OpenMP Architecture Review Board: OpenMP Application Programming Interface, Version 5.2 (2021). `https://www.openmp.org/specifications/`

[2] Ciorba, F.M., Iwainsky, C., Buder, P.: OpenMP loop scheduling revisited: Making a case for more schedules. In: IWOMP 2018, pp. 21–36 (2018). `https://doi.org/10.1007/978-3-319-98521-3_2`

[3] Müller Korndörfer, J.H., Eleliemy, A., Mohammed, A., Ciorba, F.M.: LB4OMP: A dynamic load balancing library for multithreaded applications. IEEE Trans.

Parallel Distrib. Syst. 33(4), 830–841 (2022). https://doi.org/10.1109/TPDS.2021.3107775

[4] Mohammed, A., Müller Korndörfer, J.H., Eleliemy, A., Ciorba, F.M.: Automated scheduling algorithm selection and chunk parameter calculation in OpenMP. IEEE Trans. Parallel Distrib. Syst. 33(12), 4383–4394 (2022). https://doi.org/10.1109/TPDS.2022.3189270

[5] Müller Korndörfer, J.H., Mohammed, A., Eleliemy, A., Guilloteau, Q., Krummenacher, R., Ciorba, F.M.: A comparative study of OpenMP scheduling algorithm selection strategies. IEEE Access 13, 151212–151235 (2025). https://doi.org/10.1109/ACCESS.2025.3602234

[6] Rice, J.R.: The algorithm selection problem. Advances in Computers 15, 65–118 (1976). https://doi.org/10.1016/S0065-2458(08)60520-3

[7] Watkins, C.J.C.H., Dayan, P.: Q-learning. Machine Learning 8, 279–292 (1992). https://doi.org/10.1007/BF00992698

[8] Auer, P., Cesa-Bianchi, N., Fischer, P.: Finite-time analysis of the multiarmed bandit problem. Machine Learning 47, 235–256 (2002). https://doi.org/10.1023/A:1013689704352

[9] Li, L., Chu, W., Langford, J., Schapire, R.E.: A contextual-bandit approach to personalized news article recommendation. In: WWW 2010, pp. 661–670 (2010). https://doi.org/10.1145/1772690.1772758

[10] Agrawal, S., Goyal, N.: Thompson sampling for contextual bandits with linear payoffs. In: ICML 2013. PMLR 28(3), 127–135 (2013). https://proceedings.mlr.press/v28/agrawal13.html

[11] McCalpin, J.D.: Memory bandwidth and machine balance in current high performance computers. IEEE TCCA Newsletter, 19–25 (1995). https://www.cs.virginia.edu/stream/

[12] Beamer, S., Asanović, K., Patterson, D.: The GAP Benchmark Suite. arXiv:1508.03619 (2015). https://github.com/sbeamer/gapbs

[13] DeRose, L., Homer, B., Johnson, D.: Detecting application load imbalance on high end massively parallel systems. In: Euro-Par 2007, pp. 150–159 (2007). https://doi.org/10.1007/978-3-540-74466-5_17

[14] Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction, 2nd edn. MIT Press, Cambridge (2018). http://incompleteideas.net/book/the-book-2nd.html